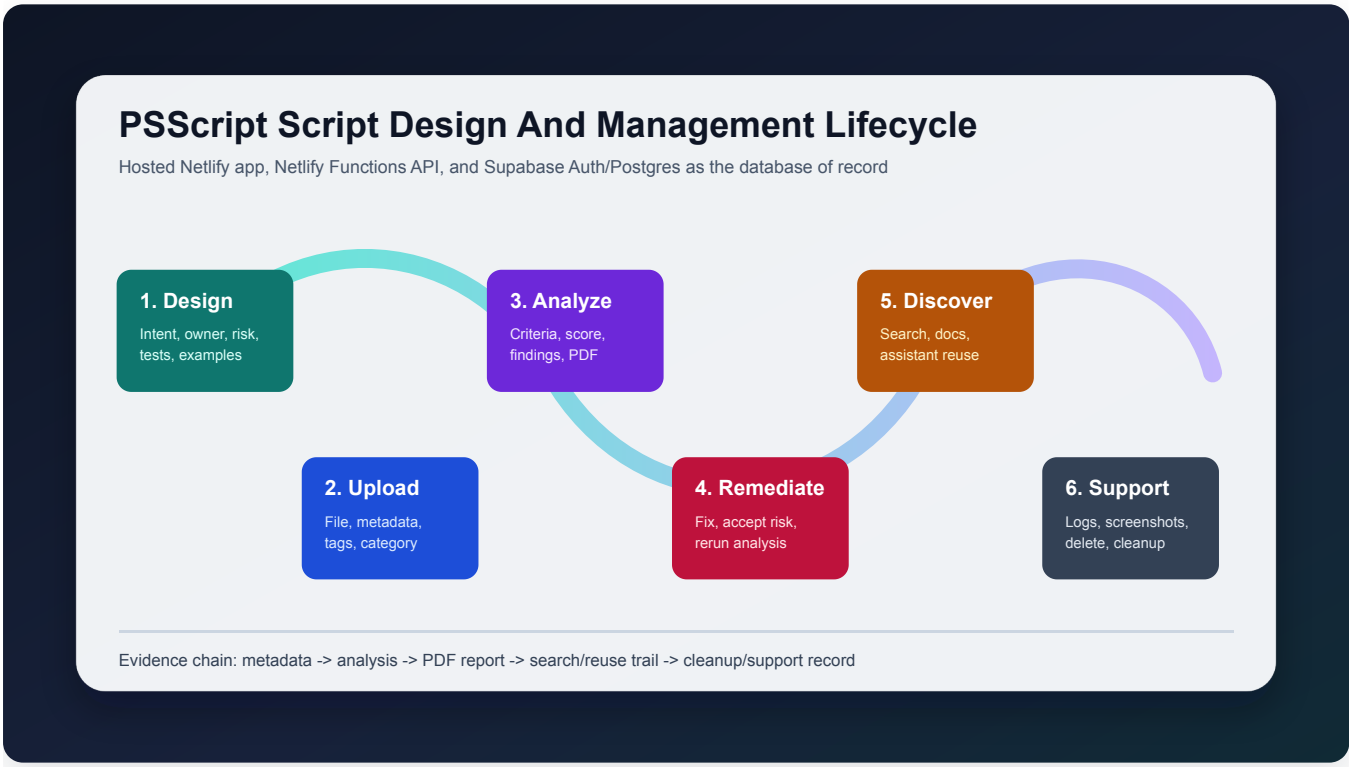
Module 02: Script Lifecycle

Last updated: April 29, 2026.

Objectives

- Design a PowerShell script with clear purpose, safety, examples, and test expectations.
- Upload scripts and validate metadata.
- Understand hash-based deduplication and versioning.
- Edit hosted script records and export the current buffer for VS Code.
- Use AI analysis and PDF export as lifecycle evidence.
- Verify delete and bulk delete behavior on safe test data.

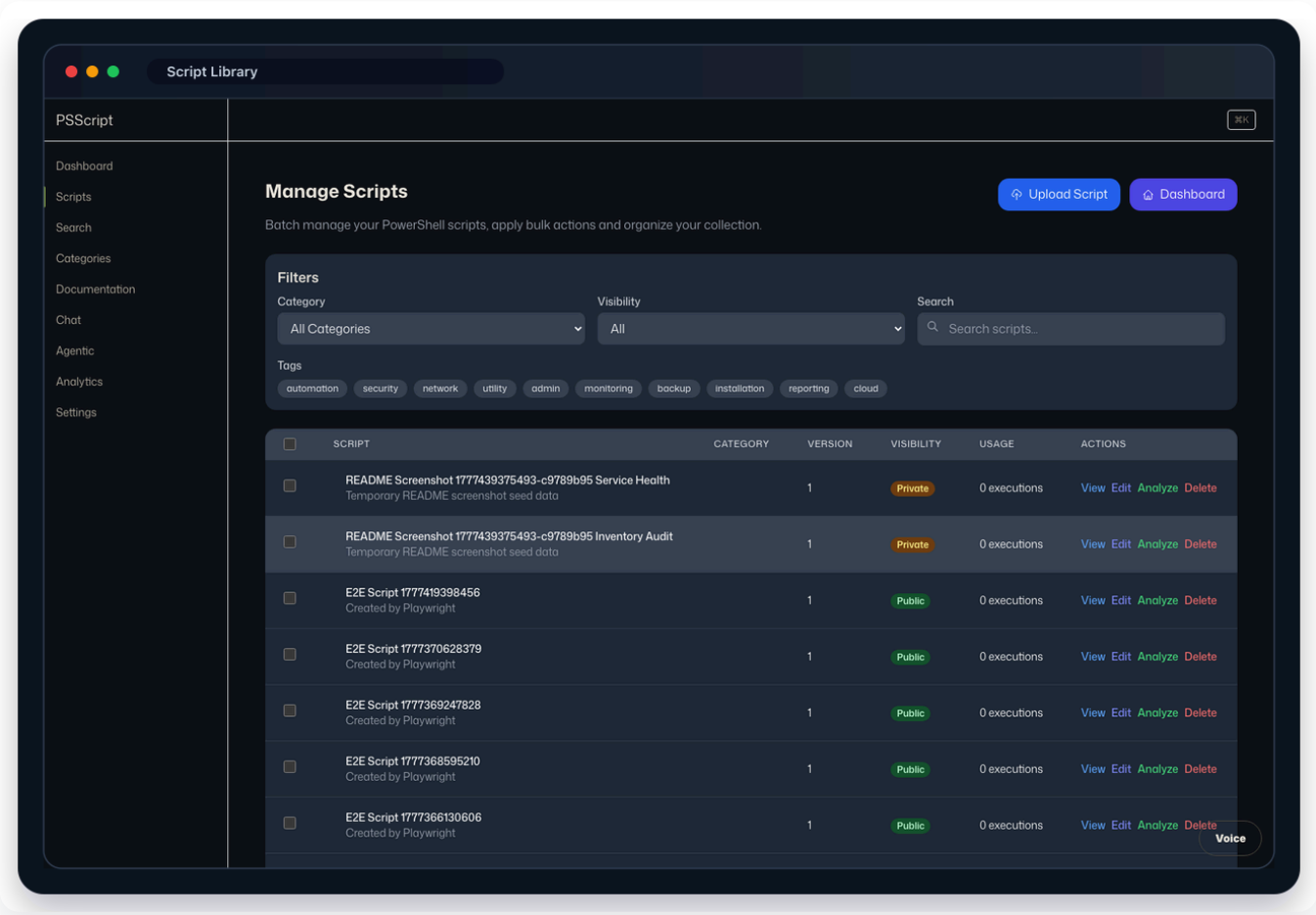


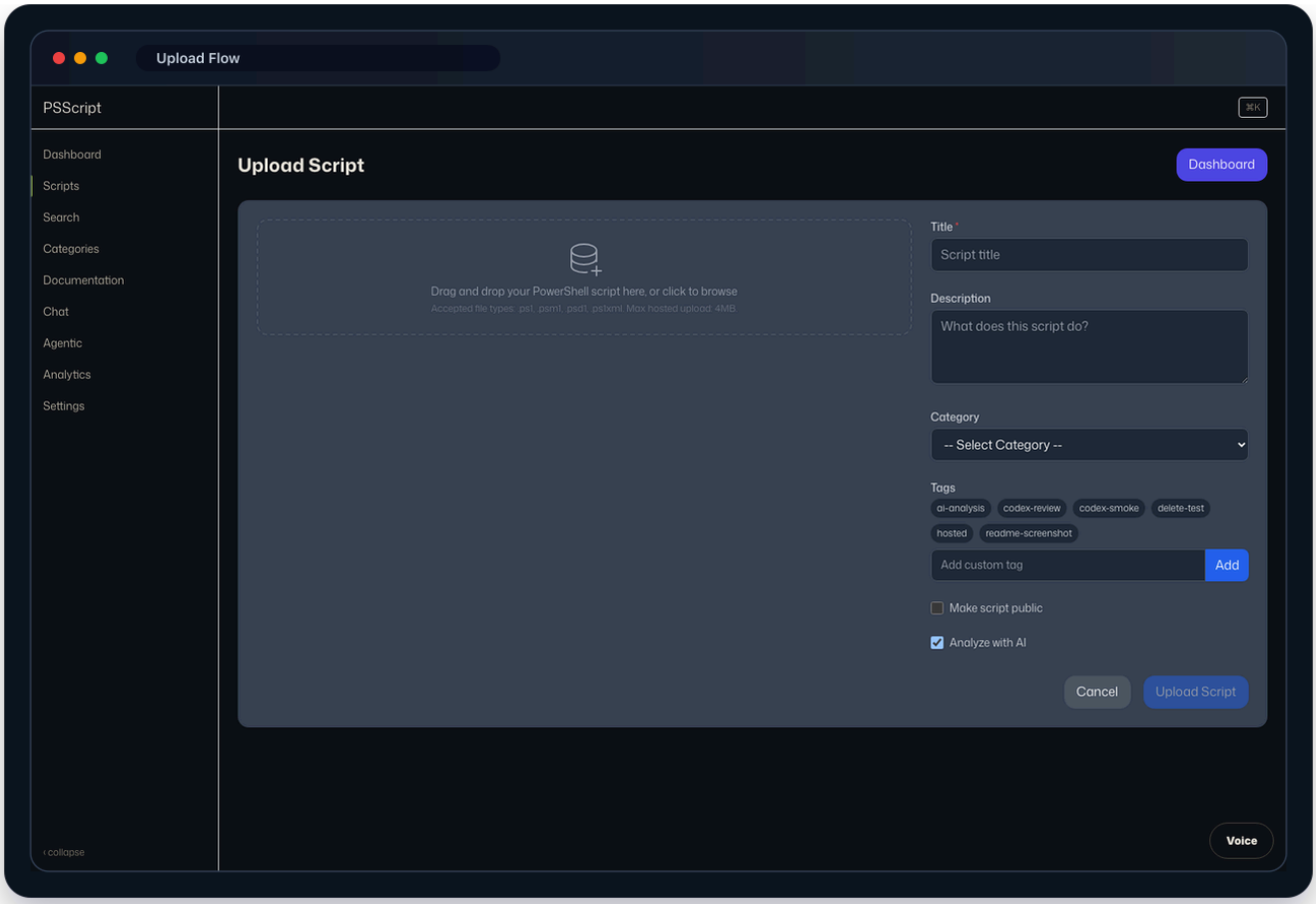
Walkthrough

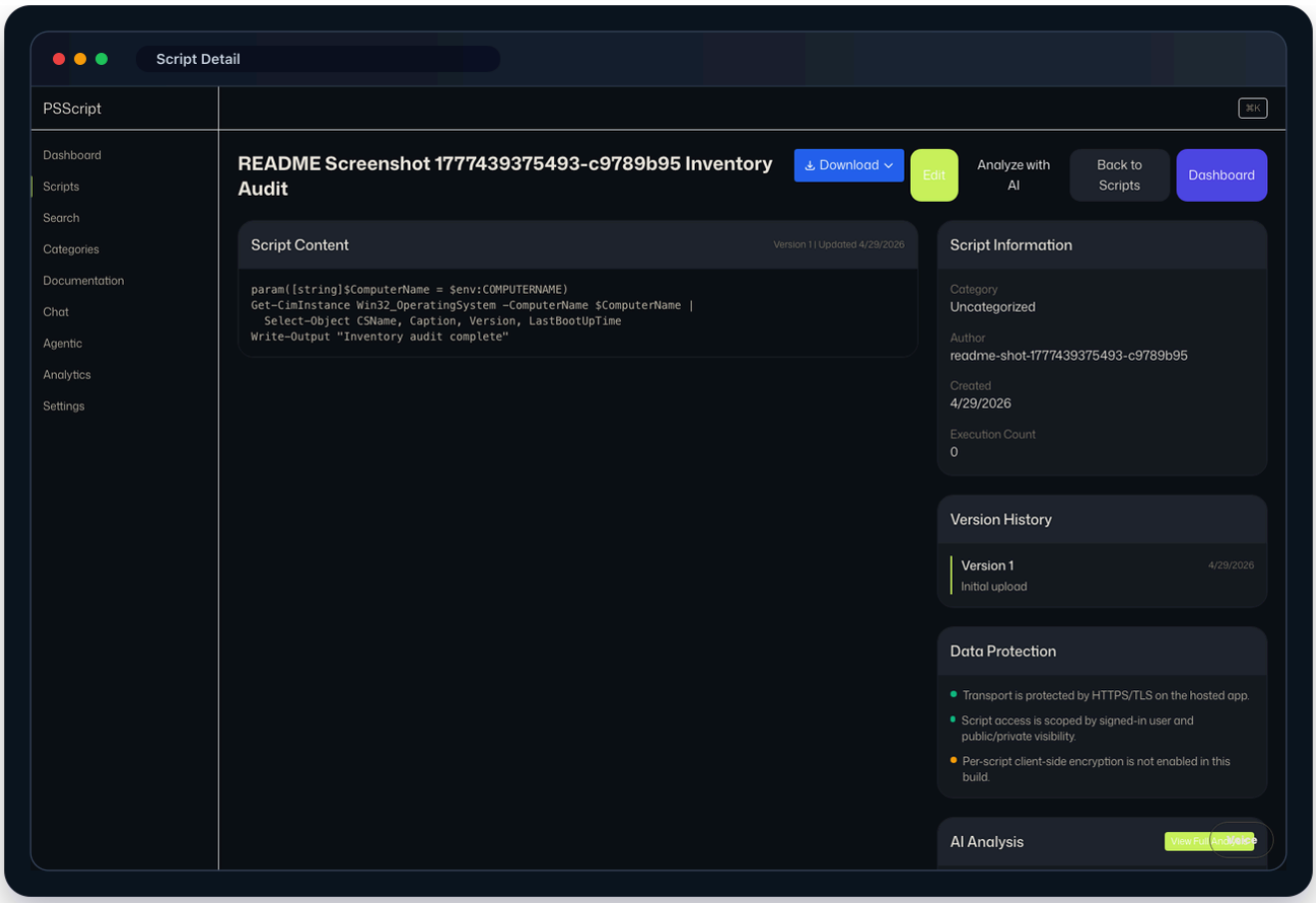
1. Define the script's intent, owner, affected system, operating environment, and expected risk.
2. Write the script with safe parameters, clear examples, and a test plan.

3. Run local/static checks when available, such as PSScriptAnalyzer for PowerShell syntax and best-practice issues.
4. Open Scripts in the hosted app.
5. Upload a safe test `.ps1` file.
6. Add title, description, tags, category, and owner metadata.
7. Open the script detail page.
8. Open the script edit page and verify title, description, and content load from hosted Supabase.
9. Use **Open in VS Code** to download the current editor buffer as a `.ps1` file for local review.
10. Review metadata, scores, version state, and analysis state.
11. Run or review AI analysis and export the PDF report.
12. For a disposable test script only, verify single delete.
13. For disposable test scripts only, verify bulk selection and bulk delete.

Screenshots




Upload Flow
PSScript
⌘K
Dashboard
Scripts
Search
Categories
Documentation
Chat
Agentic
Analytics
Settings
Upload Script
Dashboard
Drag and drop your PowerShell script here, or click to browse
Accepted file types: .ps1, .psm1, .psd1, .ps1xml. Max hosted upload: 4MB.
Title *
Script title
Description
What does this script do?
Category
-- Select Category --
Tags
ai-analysis
codex-review
codex-smoke
delete-test
hosted
readme-screenshot
Add custom tag
Add
Make script public
Analyze with AI
Cancel
Upload Script
‹ collapse
Voice


Script Detail
PSScript
⌘K
Dashboard
Scripts
Search
Categories
Documentation
Chat
Agentic
Analytics
Settings
README Screenshot 1777439375493-c9789b95 Inventory Audit
Download
Edit
Analyze with AI
Back to Scripts
Dashboard
Script Content
Version 1 | Updated 4/29/2026
param([string]$ComputerName = $env:COMPUTERNAME)
Get-CimInstance Win32_OperatingSystem -ComputerName $ComputerName |
Select-Object CSName, Caption, Version, LastBootUpTime
Write-Output "Inventory audit complete"
Script Information
Category
Uncategorized
Author
readme-shot-1777439375493-c9789b95
Created
4/29/2026
Execution Count
0
Version History
Version 1
4/29/2026
Initial upload
Data Protection
Transport is protected by HTTPS/TLS on the hosted app.
Script access is scoped by signed-in user and public/private visibility.
Per-script client-side encryption is not enabled in this build.
AI Analysis
View Full Analysis

Metadata Checklist

Field	Example	Notes
Title	Reset-UserPassword	Prefer PowerShell Verb-Noun style
Description	Reset an AD user password	Keep it concise
Category	Identity	Use current taxonomy
Tags	security, active-directory	Use meaningful tags
Owner tag	owner:team-identity	Helpful for governance
Hash	automatic	Calculated server-side

Design Checklist

Check	Why it matters
Intent is explicit	Reviewers can decide whether the script belongs in the library
Parameters are validated	Reduces accidental misuse and unsafe input
Destructive actions are guarded	Supports safer review for delete/update operations
Examples are included	Makes reuse and support faster
Tests or verification steps exist	Gives reviewers a concrete acceptance path
Secrets are externalized	Prevents credentials from entering the library
Owner and category are known	Supports future support and cleanup decisions
Runtime requirements are known	Helps reviewers confirm PowerShell version, modules, and assemblies before execution

Integrity And Deduplication

- The hosted API calculates file hashes.
- Duplicate content should be detected instead of silently creating confusing copies.
- Version history preserves prior uploads.
- Edit saves create a new version when script content changes.
- The **Open in VS Code** action downloads a `.ps1` file; it does not claim direct local filesystem access from the hosted browser.
- The hosted app uses Supabase as the database of record.
- Netlify Function payload limits are why hosted uploads are capped at 4 MB in the UI.

Verification Checklist

- Uploaded script appears in the list.
- Metadata is visible on list/detail surfaces.
- Analysis results show criteria, scores, findings, remediation, and confidence.

- Runtime requirements show PowerShell version and modules or assemblies where detected.
- Script edit export downloads a `.ps1` file.
- PDF export downloads a PDF file.
- Delete and bulk delete work only for intended test records and authorized users.

Generated 2026-04-29 14:08 UTC